



Вселенная

[Главная](#)

[Книги](#)

[О нас](#)

[Контакты](#)

[Политика конфиденциальности](#)

[DMCA](#)

Развертывание моделей машинного обучения в производственной среде

Автор: Прамод Сингх

Тип файла: pdf

Размер: 7,6 МБ

Язык: Английский

Страниц: 150

Развертывание моделей машинного обучения в рабочей среде: Flask, Streamlit, Docker, Kubernetes и Google Cloud Platform

Введение

Разработка в области машинного обучения (МО) не заканчивается, когда модель достигает высокой точности в блокноте. Настоящая инженерная задача возникает, когда эта модель должна предоставлять пользователям и приложениям надежные прогнозы. Для производственной системы МО нужны API, воспроизводимые

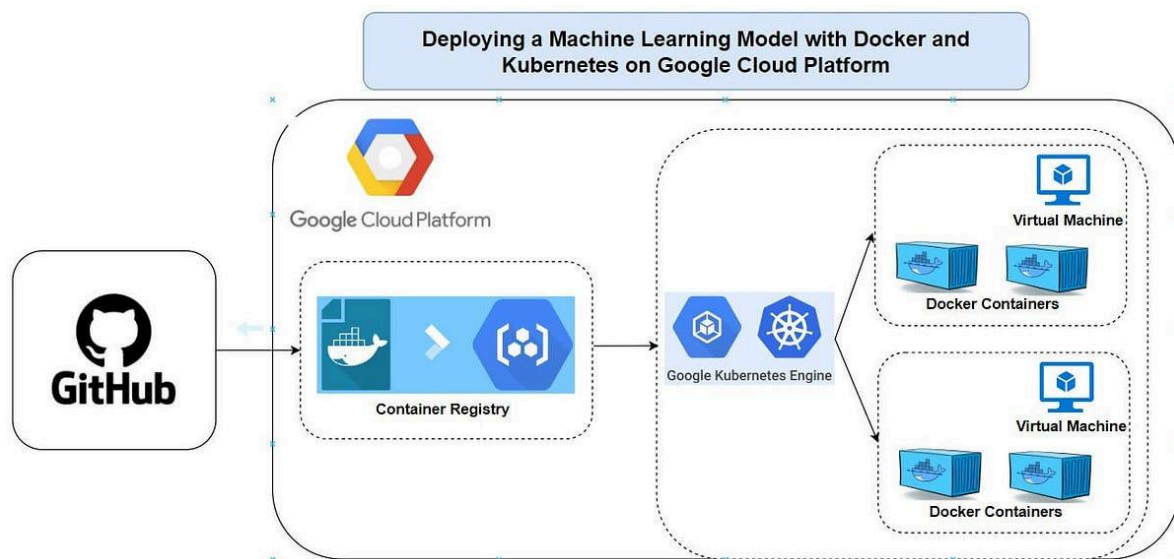
зависимости, мониторинг, масштабируемость, безопасность и стратегия развертывания.

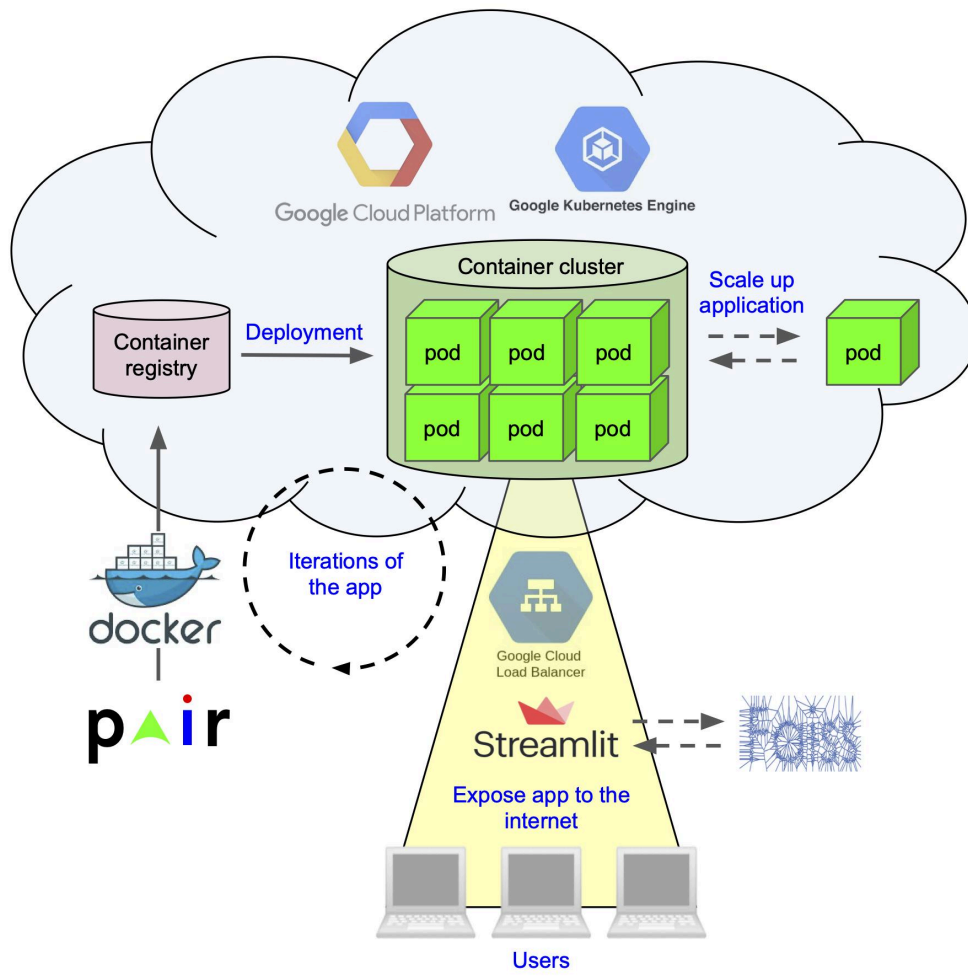
Типичный рабочий процесс может включать в себя переход от обученной модели Python к веб-интерфейсу с **Streamlit**, API логического вывода с **Flask**, портативному контейнеру с использованием **Docker** и масштабируемой системе оркестрации через **Kubernetes** на **Google Cloud Platform (GCP)**.

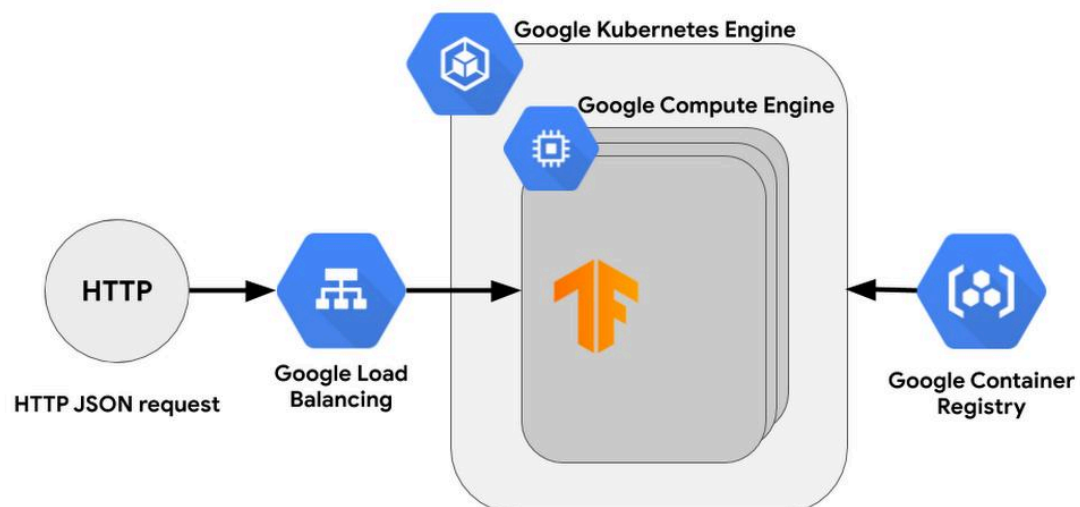
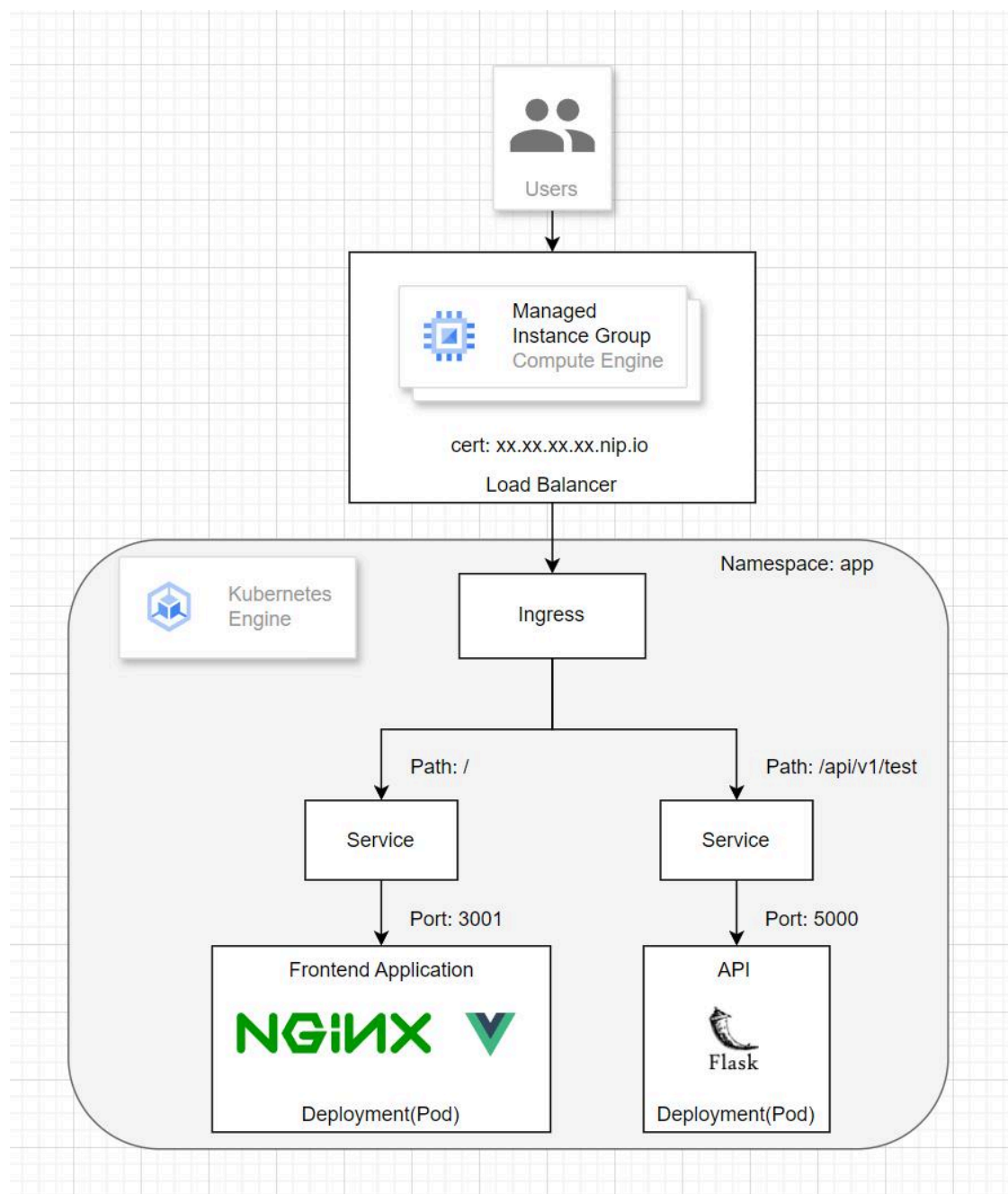
Базовый производственный процесс можно представить следующим образом:

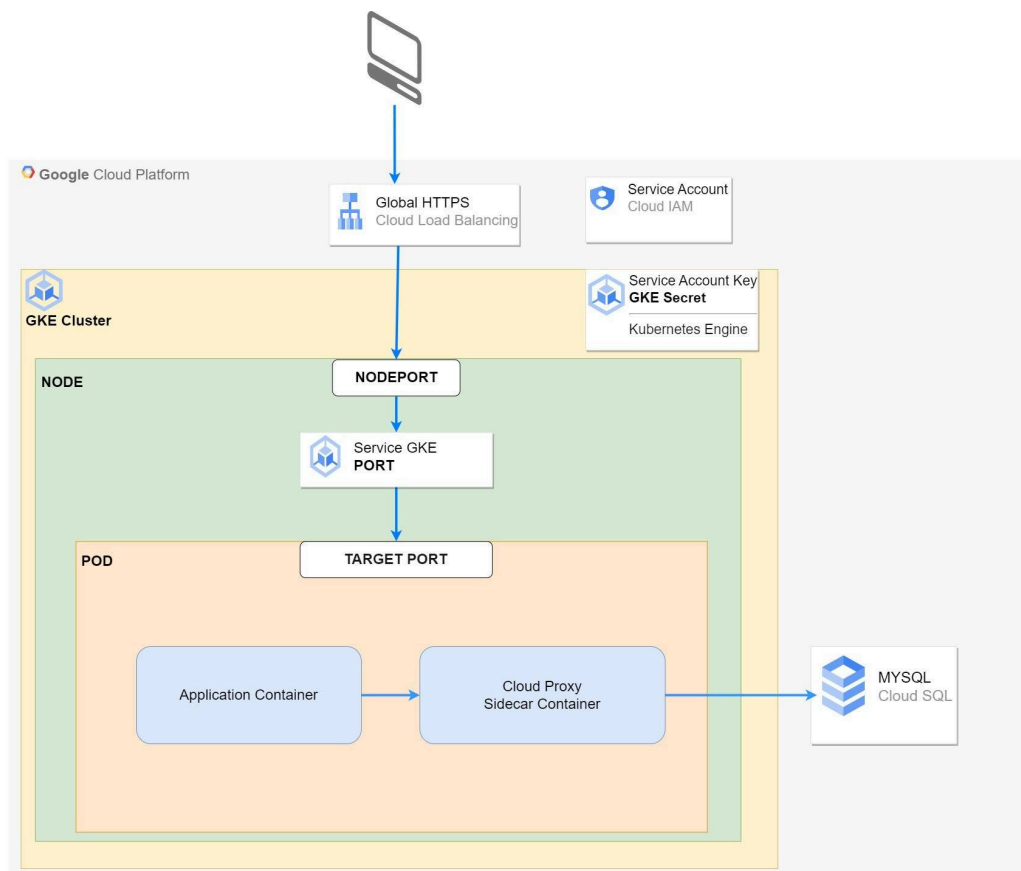
Модель → Flask API → Docker-контейнер → Kubernetes → Google Cloud → Конечный пользователь

🚀 Эта архитектура будет полезна как студентам инженерных специальностей, изучающим MLOps, так и профессионалам, создающим масштабируемые сервисы машинного обучения.









Теоретическая база

Системы машинного обучения обычно состоят из нескольких уровней:

1. **Уровень данных** — собирает и подготавливает входные данные.
2. **Уровень обучения** — обучает алгоритм машинного обучения.
3. **Уровень модели** — хранит обученную модель.
4. **Уровень приложения** — предоставляет доступ к прогнозам.
5. **Уровень инфраструктуры** — упаковывает и запускает приложение.
6. **Уровень оркестрации** — управляет контейнерами и масштабированием.
7. **Облачный уровень** — обеспечивает вычислительные ресурсы, сетевое взаимодействие, хранение данных и мониторинг.

Таким образом, производственная система существенно отличается от Jupyter Notebook.

От Notebook к производству

Рассмотрим простую модель классификации:

$$\hat{y} = f(X; \theta)$$

где:

- (X) = входные признаки
- (θ) = обученные параметры модели
- $(\hat{y})$ = прогнозируемый результат

Во время экспериментов разработчик может просто выполнить:

```
1 | prediction = model.predict(X)
```

Для производства требуется дополнительная разработка:

```
[\text{Запрос}
\rightarrow
\text{Проверка}
\rightarrow
\text{Предварительная обработка}
\rightarrow
\text{Логический вывод}
\rightarrow
\text{Ответ}]
```

Модель — это лишь один из компонентов этого конвейера.

Почему важна контейнеризация

Модель может отлично работать на компьютере разработчика, но давать сбой в других условиях из-за:

- Различия в версиях Python
- несовместимые версии пакетов
- отсутствующие системные библиотеки
- неправильные переменные среды
- различные зависимости от операционной системы

Docker addresses this problem by packaging the application and its dependencies into a reproducible container.

Definition

Machine learning model deployment is the process of making a trained ML model available to an application, service, user, or automated system so that it can generate predictions using new data.

A production ML deployment can be defined as:

[D = M + API + C + O + Cloud]

where:

- (M) = machine learning model
- (API) = application programming interface
- (C) = containerization
- (O) = orchestration
- Cloud = cloud infrastructure

Flask

Flask is a lightweight Python web framework that can expose a machine learning model through HTTP endpoints.

For example:

```
1 | POST /predict
```

could receive:

```
1 | {
2 |   "age": 35,
3 |   "income": 72000,
4 |   "experience": 8
5 | }
```

and return:

```
1 | {
2 |   "prediction": 1,
3 |   "probability": 0.91
4 | }
```

Streamlit

Streamlit provides a convenient way to build interactive ML interfaces using Python.

Instead of sending JSON manually, a user could interact with:

- text fields
- sliders
- dropdown menus

- buttons
- charts

 Streamlit is particularly useful for demonstrations, internal tools, prototypes, and data applications.

Docker

Docker packages the application into an image that can run consistently across environments.

Kubernetes

Kubernetes manages containerized applications and provides capabilities such as:

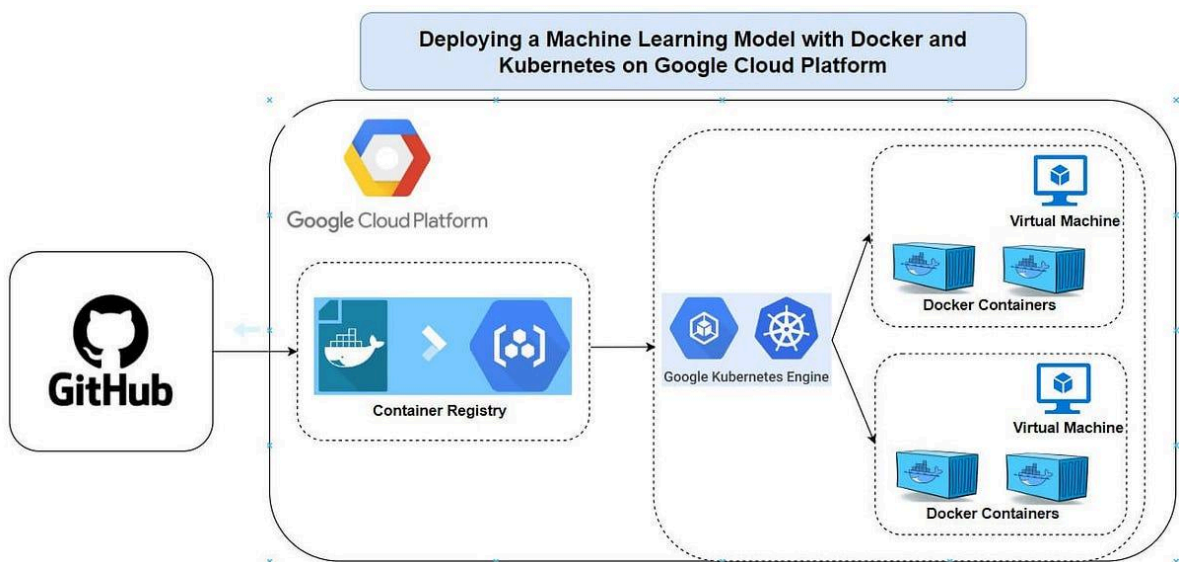
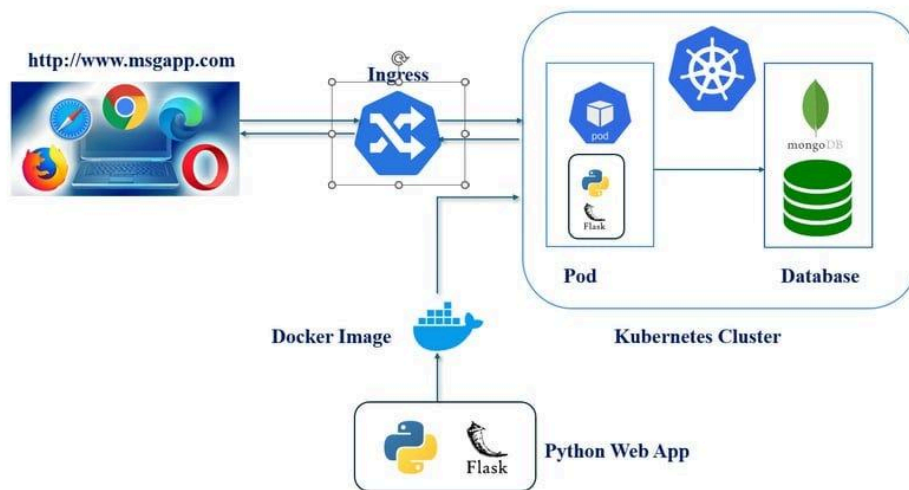
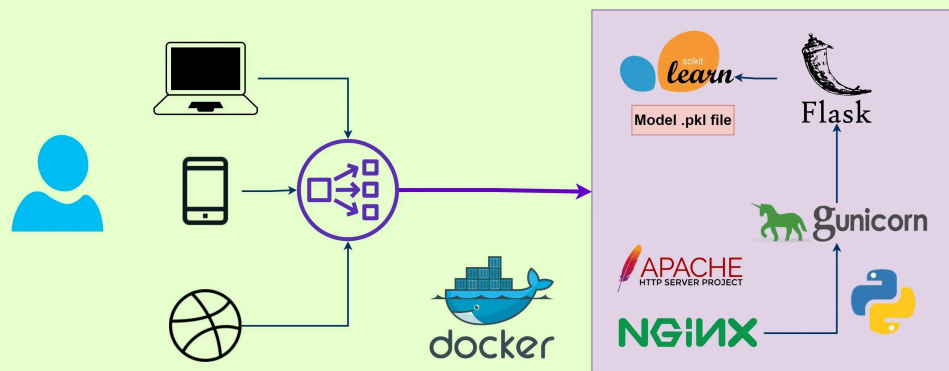
- deployment
- scaling
- service discovery
- rolling updates
- workload management
- self-healing

Google Cloud Platform

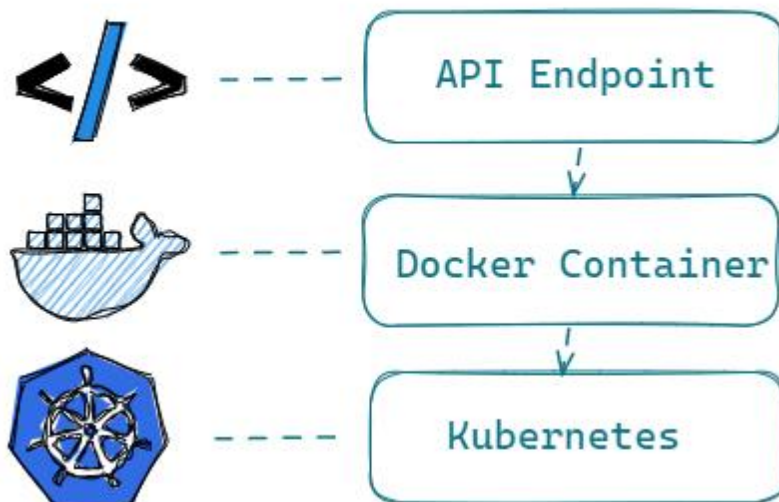
GCP provides cloud infrastructure on which containers, Kubernetes clusters, storage systems, networking, and monitoring tools can operate.

Step-by-Step Machine Learning Deployment

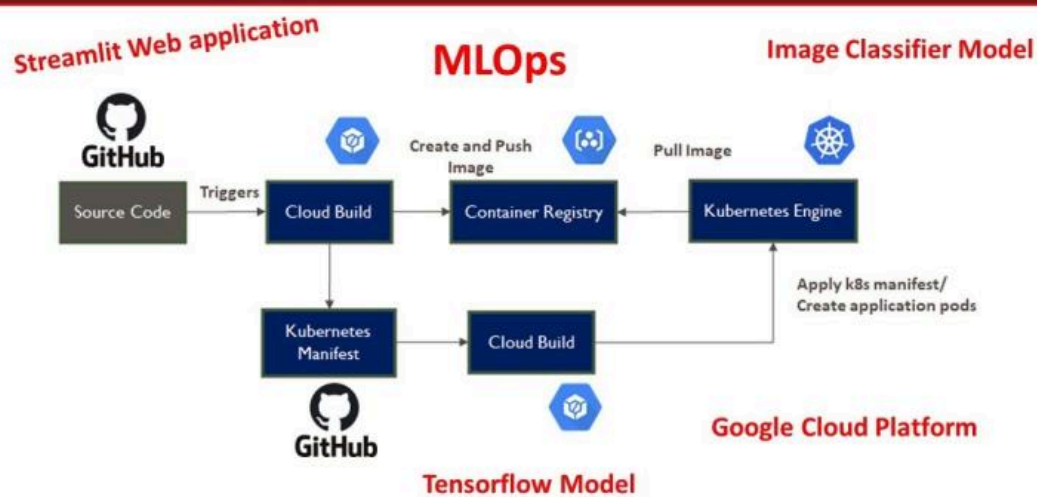
Machine Learning API Development & Integration



Online Model Deployment Patterns



Continuous Deployment - GitHub and Kubernetes



Step 1: Train and Save the Model

Suppose we train a simple scikit-learn model.

```
1 | from sklearn.ensemble import RandomForestClassifier
2 | import joblib
3 |
4 | model = RandomForestClassifier(
5 |     n_estimators=100,
6 |     random_state=42
7 | )
8 |
9 | model.fit(X_train, y_train)
10 |
11 | joblib.dump(model, "model.pkl")
```

The resulting file might be:

```
1 | model.pkl
```

The model should ideally be accompanied by the exact preprocessing pipeline used during training.

Step 2: Build the Flask API

Create:

```
1 | app.py
```

A simplified API might look like:

```
1 | from flask import Flask, request, jsonify
2 | import joblib
3 |
4 | app = Flask(__name__)
5 |
6 | model = joblib.load("model.pkl")
7 |
8 | @app.route("/predict", methods=["POST"])
9 | def predict():
10 |     data = request.get_json()
11 |
12 |     features = [[
13 |         data["age"],
14 |         data["income"],
15 |         data["experience"]
16 |     ]]
17 |
18 |     prediction = model.predict(features)[0]
19 |     probability = model.predict_proba(features)[0].max()
20 |
21 |     return jsonify({
22 |         "prediction": int(prediction),
23 |         "probability": float(probability)
24 |     })
25 |
26 | @app.route("/health", methods=["GET"])
27 | def health():
28 |     return jsonify({"status": "healthy"})
29 |
30 |
```

```
31 | if __name__ == "__main__":  
    app.run(host="0.0.0.0", port=8080)
```

The /health endpoint is especially important for container orchestration.

Step 3: Add Dependencies

Create:

```
1 | requirements.txt
```

For example:

```
1 | Flask  
2 | scikit-learn  
3 | joblib  
4 | gunicorn
```

For production, package versions should normally be pinned or otherwise controlled to improve reproducibility.

Step 4: Test the API Locally

Run:

```
1 | python app.py
```

Then send a request:

```
1 | curl -X POST http://localhost:8080/predict \  
2 | -H "Content-Type: application/json" \  
3 | -d '{"age":35,"income":72000,"experience":8}'
```

A successful response could look like:

```
1 | {  
2 |   "prediction": 1,  
3 |   "probability": 0.91  
4 | }
```

Step 5: Create a Dockerfile

```
1 | FROM python:3.11-slim
2 |
3 | WORKDIR /app
4 |
5 | COPY requirements.txt .
6 |
7 | RUN pip install --no-cache-dir -r requirements.txt
8 |
9 | COPY model.pkl .
10 | COPY app.py .
11 |
12 | EXPOSE 8080
13 |
14 | CMD ["gunicorn", "--bind", "0.0.0.0:8080", "app:app"]
```

The resulting architecture is:

```
1 | Client
2 |   |
3 |   ▼
4 | Flask API
5 |   |
6 |   ▼
7 | ML Model
8 |   |
9 |   ▼
10 | Prediction
```

Step 6: Build the Docker Image

```
1 | docker build -t ml-api:v1 .
```

Run it:

```
1 | docker run -p 8080:8080 ml-api:v1
```

Now the model is running inside a container.

Step 7: Add a Streamlit Interface

A simple interface might be:

```

1 | import streamlit as st
2 | import requests
3 |
4 | st.title("🤖 ML Prediction System")
5 |
6 | age = st.number_input("Age", 18, 100, 35)
7 | income = st.number_input("Income", 0, 500000, 72000)
8 | experience = st.number_input("Experience", 0, 50, 8)
9 |
10 | if st.button("Predict"):
11 |     payload = {
12 |         "age": age,
13 |         "income": income,
14 |         "experience": experience
15 |     }
16 |
17 |     response = requests.post(
18 |         "http://localhost:8080/predict",
19 |         json=payload
20 |     )
21 |
22 |     result = response.json()
23 |
24 |     st.success(f"Prediction: {result['prediction']}")
25 |     st.info(f"Confidence: {result['probability']:.2%}")

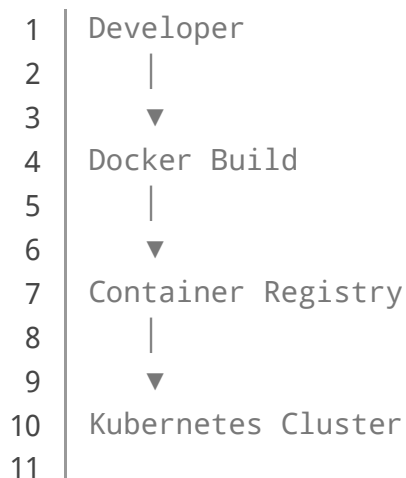
```

The Streamlit application becomes the presentation layer while Flask remains responsible for inference.

Step 8: Push the Container to Google Cloud

A production workflow can use **Artifact Registry** to store container images.

Conceptually:



```
12 |      |
13 |      ▼
    ML API
```

Before deploying to GCP, configure the appropriate project, region, authentication, APIs, repository, and IAM permissions.

Step 9: Deploy with Kubernetes

A simplified Kubernetes Deployment could look like:

```
1  apiVersion: apps/v1
2  kind: Deployment
3  metadata:
4    name: ml-api
5  spec:
6    replicas: 3
7    selector:
8      matchLabels:
9        app: ml-api
10   template:
11     metadata:
12       labels:
13         app: ml-api
14     spec:
15       containers:
16         - name: ml-api
17           image: REGION-docker.pkg.dev/PROJECT_ID/ml-repo/ml-api
18           ports:
19             - containerPort: 8080
20           resources:
21             requests:
22               cpu: "250m"
23               memory: "512Mi"
24             limits:
25               cpu: "1"
26               memory: "1Gi"
```

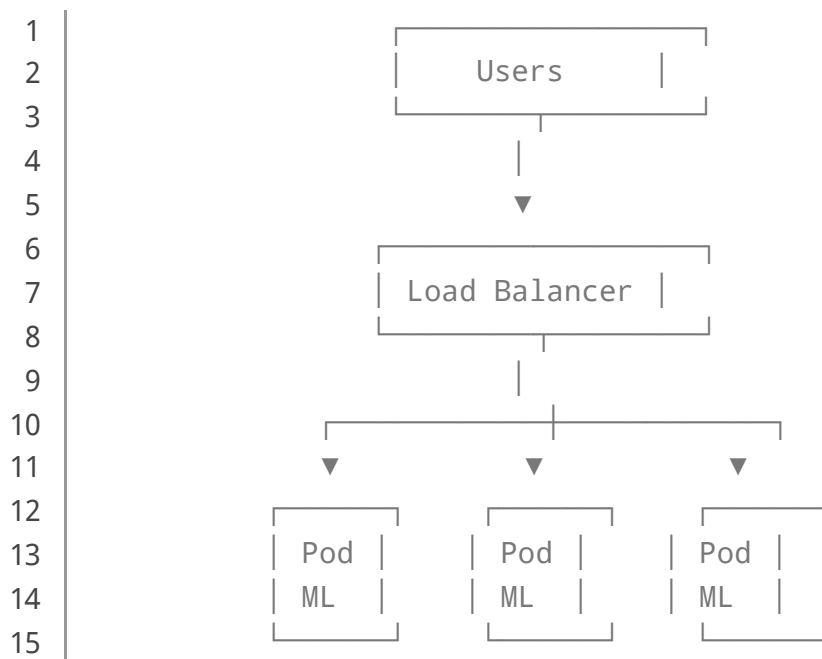
Three replicas mean Kubernetes attempts to maintain three running instances.

Step 10: Expose the Application

A Kubernetes Service can expose the deployment:

```
1 | apiVersion: v1
2 | kind: Service
3 | metadata:
4 |   name: ml-api-service
5 | spec:
6 |   selector:
7 |     app: ml-api
8 |   ports:
9 |     - port: 80
10 |       targetPort: 8080
11 |   type: LoadBalancer
```

The resulting architecture becomes:



Comparison

Technology	Primary Role	Main Advantage	Typical Use
Flask	API	Lightweight and flexible	ML inference API
Streamlit	UI	Very fast development	ML demos and dashboards
Docker	Packaging	Reproducibility	Container deployment

Technology	Primary Role	Main Advantage	Typical Use
Kubernetes	Orchestration	Scaling and resilience	Production workloads
GCP	Cloud infrastructure	Managed cloud services	Enterprise deployment

Flask vs. Streamlit

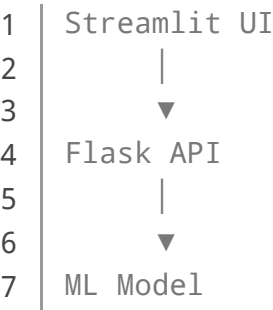
Flask is primarily an API framework.

Streamlit is primarily an application/UI framework.

Therefore:

$\text{Flask} \neq \text{Streamlit}$

They can instead complement each other.



Docker vs. Kubernetes

Docker answers:

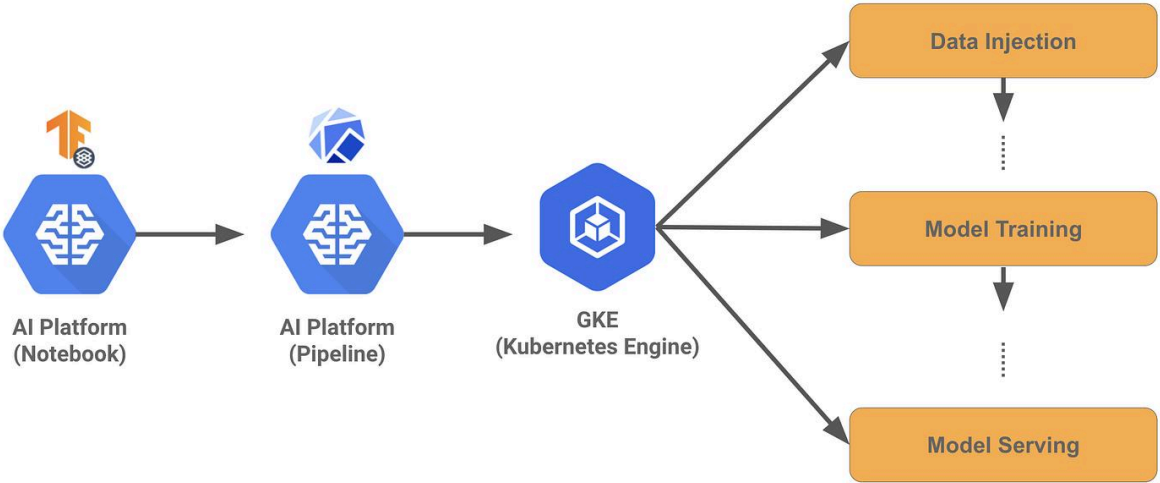
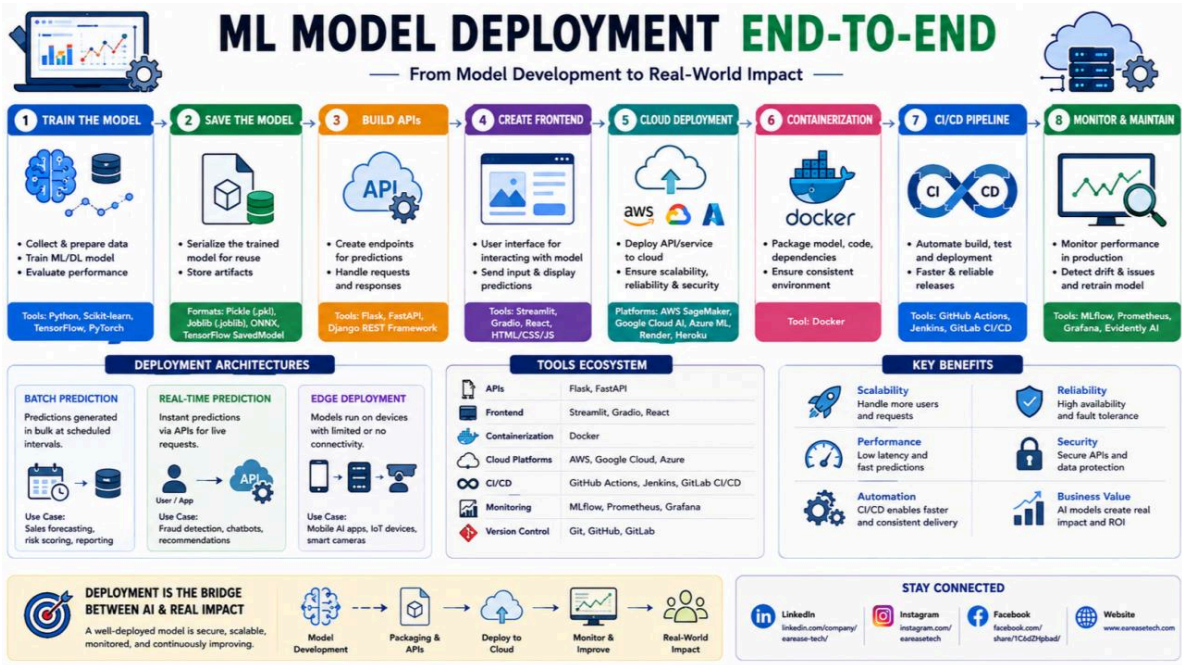
How do I package and run my application consistently?

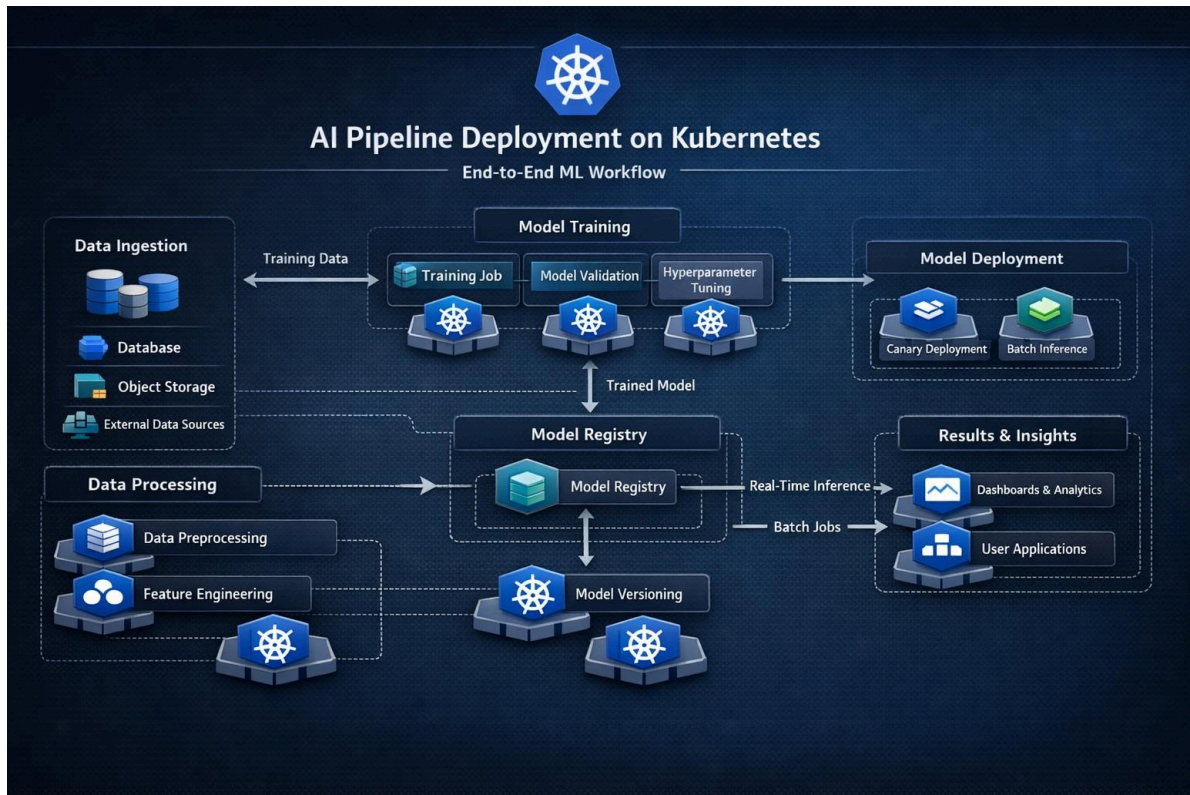
Kubernetes answers:

How do I manage many running containers reliably?

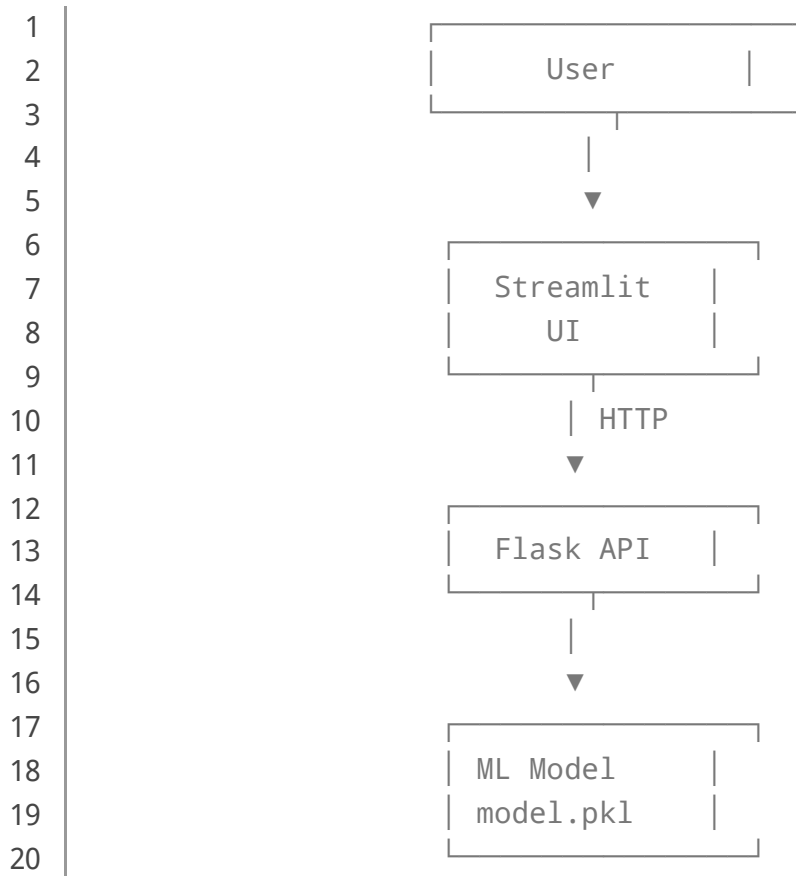
This distinction is fundamental for understanding modern cloud-native ML systems.

Diagrams and Tables

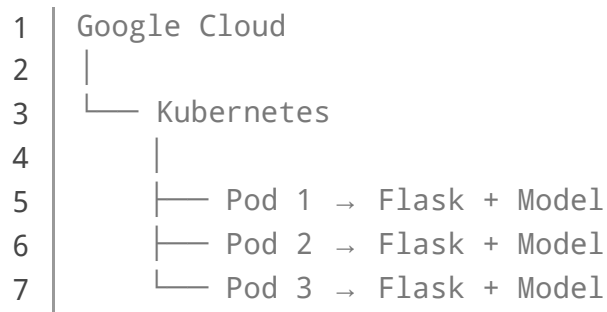




Production Architecture



Containerized Architecture



Key Production Metrics

Metric	Meaning	Why It Matters
Latency	Time per prediction	User experience
Throughput	Requests/second	Capacity
Error Rate	Failed requests	Reliability
CPU Usage	Processor consumption	Scaling
Memory Usage	RAM consumption	Stability
Model Accuracy	Prediction quality	ML performance
Drift	Change in input/output behavior	Long-term reliability

Examples

Example 1: Customer Classification

An e-commerce company can deploy a model predicting whether a customer is likely to purchase.

Input:

$X = [\text{Age}, \text{Income}, \text{Visits}, \text{CartValue}]$

Output:

$P(\text{Purchase}=1|X)=0.87$

The Flask API receives the customer information and returns the probability.

Example 2: Engineering Predictive Maintenance

A manufacturing system can use:

`[X=[Temperature,Vibration,Pressure,Speed]]`

to estimate machine failure probability.

For example:

```
1 | {  
2 |   "temperature": 83.4,  
3 |   "vibration": 7.2,  
4 |   "pressure": 4.8,  
5 |   "speed": 1800  
6 | }
```

The model might return:

```
1 | {  
2 |   "failure_probability": 0.78  
3 | }
```

⚙️ Engineers can then trigger inspection before catastrophic equipment failure.

Example 3: Financial Risk Classification

A model can estimate a risk category from:

- transaction history
- account characteristics
- customer behavior
- payment patterns

The API can serve the model to multiple internal applications without embedding the ML model into each application.

Real-World Application

Production ML deployment is particularly valuable when prediction must be consumed repeatedly.

Healthcare Systems

ML APIs can support medical image classification, risk prediction, or operational forecasting. Such systems require strict validation, privacy, security, and human oversight.

Manufacturing

Predictive-maintenance models can process sensor data continuously.

A simplified relationship is:

$$[\text{Risk} = f(T, V, P, S)]$$

where:

- (T) = temperature
- (V) = vibration
- (P) = pressure
- (S) = speed

Transportation

Models can estimate:

- travel time
- traffic congestion
- fuel consumption
- maintenance requirements

E-Commerce

Production models can power:

- recommendation engines
- customer segmentation
- demand forecasting
- fraud detection
- search ranking

Engineering Design

ML models can be exposed through web applications so engineers can evaluate designs without directly interacting with the underlying Python environment.

Common Mistakes

Mistake 1: Deploying the Notebook Directly

A notebook is an experimentation environment, not necessarily a production service.

Solution: Separate training, inference, API, and infrastructure components.

Mistake 2: Ignoring Preprocessing

Suppose training uses:

$$[X' = \frac{X - \mu}{\sigma}]$$

but production sends raw (X).

The model may produce unreliable predictions.

Solution: Package preprocessing with the model pipeline.

Mistake 3: Using Development Servers

Flask's development server should not normally be treated as a production application server.

Solution: Use a production WSGI server such as Gunicorn behind an appropriate production architecture.

Mistake 4: No Health Endpoint

Kubernetes needs to know whether an application is functioning.

Solution:

```
1 | GET /health
```

should return a meaningful health response.

Mistake 5: No Resource Limits

A model can consume excessive CPU or memory.

Solution: Define Kubernetes resource requests and limits.

Mistake 6: Hard-Coding Secrets

Never place API keys or credentials directly into source code.

 Use appropriate secret-management mechanisms and IAM controls.

Challenges and Solutions

Challenge	Potential Solution
High latency	Optimize model and infrastructure
Memory exhaustion	Reduce model size or increase resources
Traffic spikes	Horizontal scaling
Model drift	Monitor production data
Version conflicts	Docker
Deployment failures	CI/CD and staged releases
Security risks	IAM, TLS, secrets management
Poor observability	Logging and monitoring

Scaling

If one container processes (R) requests/second and traffic reaches (T) requests/second, a rough initial replica estimate is:

$$[N \geq \frac{T}{R}]$$

For example, if:

$$[T=300 \text{ requests/s}]$$

and each replica handles approximately:

$$[R=100 \text{ requests/s}]$$

then:

$$[N \geq 3]$$

In real systems, additional capacity should normally be reserved for variability and failures.

Case Study

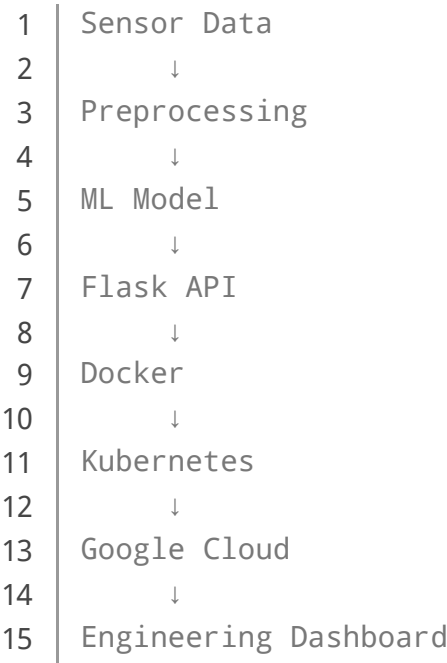
Production Predictive-Maintenance Platform

Consider a hypothetical industrial company operating hundreds of machines.

The engineering team trains a failure-prediction model using historical sensor measurements.

The original model runs inside a Python notebook.

The team converts the system into:



Deployment Strategy

The team initially deploys three replicas.

Each replica contains:

- 1 | Python
- 2 | Flask
- 3 | Gunicorn
- 4 | Model
- 5 | Dependencies

Kubernetes distributes requests between replicas.

If traffic increases, additional replicas can be created.

If one pod fails, Kubernetes can replace it.

Engineering Outcome

The main improvement is not simply model accuracy.

The organization gains:

- repeatable deployment
- centralized inference
- scalable infrastructure
- controlled versions
- easier monitoring
- improved operational reliability

This illustrates a fundamental MLOps principle:

A production ML model is a software system, not merely a trained algorithm.

Essential Tips

1. Version Everything

Version:

- source code
- Docker images
- models
- configuration
- dependencies

For example:

```
1 | ml-api:v1
2 | ml-api:v2
3 |
```

2. Keep Training Separate from Inference

Training may require GPUs and large datasets.

Inference may require low latency and predictable resource consumption.

Separating them often produces a cleaner architecture.

3. Monitor the Model, Not Just the Server

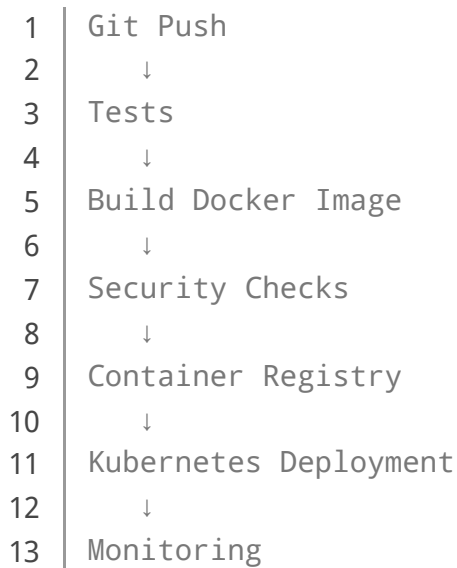
CPU utilization can be normal while model quality deteriorates.

Monitor:

$[\text{System Health}] + [\text{Data Quality}] + [\text{Model Quality}]$

4. Automate Deployment

A mature workflow can be:



5. Design for Failure

Production infrastructure should assume that:

- containers can crash
- networks can fail
- dependencies can become unavailable
- traffic can suddenly increase

Resilience should therefore be designed rather than assumed.

FAQs

What is the difference between Flask and Streamlit?

Flask is commonly used to build APIs and backend services, while Streamlit is designed primarily for interactive Python-based web applications and data/ML interfaces. They can be used together.

Why should I use Docker for an ML model?

Docker packages the model, application, runtime, and dependencies into a reproducible environment. This significantly reduces environment-related deployment problems.

Do I need Kubernetes for every [ML project](#)?

No. A small prototype may not need Kubernetes. Kubernetes becomes more valuable when you require orchestration, multiple replicas, automated scaling, resilience, and more sophisticated deployment management.

Why use Google Cloud Platform?

GCP provides cloud infrastructure and managed services that can support scalable ML applications, container registries, Kubernetes environments, networking, storage, monitoring, and security.

Should the model be inside the Docker image?

For relatively small and stable models, packaging the model with the application image can be practical. For larger or frequently updated models, external model storage and controlled model loading may be preferable.

How can I scale an ML API?

You can increase the number of application replicas and distribute requests among them. Kubernetes can help automate this process based on appropriate workload and resource metrics.

How do I monitor a deployed ML model?

Monitor infrastructure metrics such as CPU, memory, latency, and errors, but also monitor ML-specific metrics such as prediction distributions, data quality, drift, and—when labels become available—model performance.

Is Streamlit suitable for enterprise production?

It can be useful for internal applications, dashboards, demonstrations, and certain production workloads. However, applications requiring complex APIs, highly customized frontends, or large-scale public traffic may benefit from a more specialized frontend/backend architecture.

Conclusion

Deploying a machine learning model to production requires much more than saving a `.pkl` file and calling `predict()`.

A practical production architecture can combine **Flask** for inference APIs, **Streamlit** for interactive interfaces, **Docker** for reproducible packaging, **Kubernetes** for orchestration, and **Google Cloud Platform** for scalable cloud infrastructure.

The complete engineering pipeline can be summarized as:

```
[\boxed{
Model
\rightarrow
API
\rightarrow
Container
\rightarrow
Orchestration
\rightarrow
Cloud
\rightarrow
Monitoring}]
```

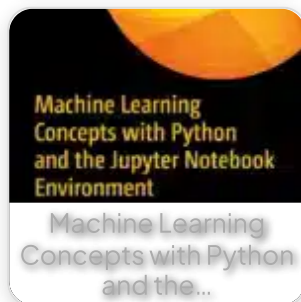
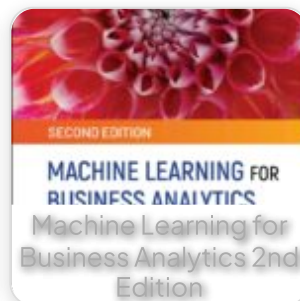
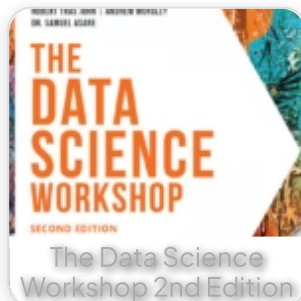
 The most important lesson is that successful ML deployment sits at the intersection of **machine learning, software engineering, DevOps, cloud computing, and systems engineering**.

For students, this workflow provides a practical path from an ML experiment to a deployable application. For professionals, it provides the foundation for building systems that can handle real users, changing workloads, software updates, and production failures.

Ultimately, the goal is not simply to make a model **predict**.

The goal is to make it **reliable, scalable, observable, maintainable, secure, and useful in the real world**.

Related Posts:



[Preview PDF Book](#)

[← Previous Book](#)

[Next Book →](#)

Explore a vast library of free engineering ebooks curated to help you expand your technical knowledge, conduct groundbreaking research, and advance your career. Our mission is to create a hub of knowledge that fosters innovation, empowers aspiring engineers, and bridges the gap between education and industry.